SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA
(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DATASCIENCE TOOLKIT LAB RECORD

Name : T.S.S.S.BRAHMAJI

Reg. No : **11249A369**

Class : **II- B.E.(CSE)**

Subject :**BCSF244AI2**
DATASCIENCE TOOLKIT LAB

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by
Mr/Ms. T.S.S.S.BRAHMAJI

with **Reg. No:** 11249A369 of **II-B.E.(CSE)** in **DATASCIENCE TOOLKIT LAB** (**BCSF244AI2)** during the academic year 2025 – 2026.

Station: Enathur

Date:

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

INDEX

SL.NO	NAME OF THE EXPERIMENT	DATE	SIGN
1	Introduction to python libraries-NumPy,Pandas,Mataplotib,Scikit-learn	19/02/26	
2	Perform data exploration and preprocessing Python	09/02/26	
3	Implement Regularized Linear Regression	14/02/26	
4	Implement Naïve Bayes classifier	16/02/26	
5	Implement Regularized Logistic regression	23/02/26	
6	Bulid models using ensembling techniques	02/03/26	
7	Bulid models using decision tree	16/03/26	
8	Support vector machine with different Kernels	23/03/26	
9	Implement K-Nearest Neighbours Classification	30/03/26	
10	K-Means clustering with PCA and elbow Method	06/04/26	

EXPERIMENT.NO:01

INTRODUCTION TO PYTHON LIBRARIES FOR DATA SCIENCE

AIM : To Introduce python libraries for data science .

PROGRAM :
PANDAS LIBRARY

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Paris', 'London']
}
df = pd.DataFrame(data)

print(df.head())
print(df.info())
print(df.describe())
#T.SIDDARTHA
#REG.NO:11249A373
```

```
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
        Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
```

```python
ages = df['Age']
print(ages)
above_25 = df[df['Age'] > 25]
print(above_25)
row_0 = df.iloc[0]
print(row_0)
specific_val = df.loc[0, 'City']
print(specific_val)
#T.SIDDARTHA
#REG.NO:11249A373
```

```
0     25
1     30
2     35
Name: Age, dtype: int64
      Name  Age     City
1      Bob   30    Paris
2  Charlie   35   London
Name        Alice
Age            25
City     New York
Name: 0, dtype: object
New York
```

```
print(df.isnull().sum())
df_clean = df.dropna()
df_filled = df.fillna(0)

#T.SIDDARTHA
#REG.NO:11249A373
```

```
Name     0
Age      0
City     0
dtype: int64
```

NUMPY LIBRAARY

```python
import numpy as np

print(
    "1D Array:\n", np.array([1,2,3,4,5]), "\n\n"
    "2D Array:\n", np.array([[1,2,3],[4,5,6]]), "\n\n"
    "Zeros:\n", np.zeros((3,3)), "\n\n"
    "Ones:\n", np.ones((2,4)), "\n\n"
    "Identity:\n", np.eye(3), "\n\n"
    "Range:\n", np.arange(0,10,2)
    )
#T.SIDDARTHA
#REG.NO:11249A373
```

```
1D Array:
 [1 2 3 4 5]

2D Array:
 [[1 2 3]
 [4 5 6]]

Zeros:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

Ones:
 [[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

Identity:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Range:
 [0 2 4 6 8]
```

```python
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(
    "Array a:", a, "\n"
    "Array b:", b, "\n\n"
    "Addition (a + b):", a + b, "\n"
    "Multiplication (a * b):", a * b, "\n"
    "Scalar Multiplication (a * 10):", a * 10, "\n\n"
    "Square Root of a:", np.sqrt(a), "\n"
    "Mean of a:", np.mean(a), "\n"
    "Dot Product (a · b):", np.dot(a, b)
    )
#T.SIDDARTHA
#REG.NO:11249A373
```

```
Array a: [1 2 3]
Array b: [4 5 6]

Addition (a + b): [5 7 9]
Multiplication (a * b): [ 4 10 18]
Scalar Multiplication (a * 10): [10 20 30]

Square Root of a: [1.         1.41421356 1.73205081]
Mean of a: 2.0
Dot Product (a · b): 32
```

```python
import numpy as np

matrix = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])

print(
    "Matrix:\n", matrix, "\n\n"
    "Element at [1,2]:", matrix[1, 2], "\n\n"
    "First Row:", matrix[0, :], "\n"
    "Second Column:", matrix[:, 1], "\n\n"
    "Top-left 2x2 Sub-matrix:\n", matrix[0:2, 0:2]
    )
#T.SIDDARTHA
#REG.NO:11249A373
```

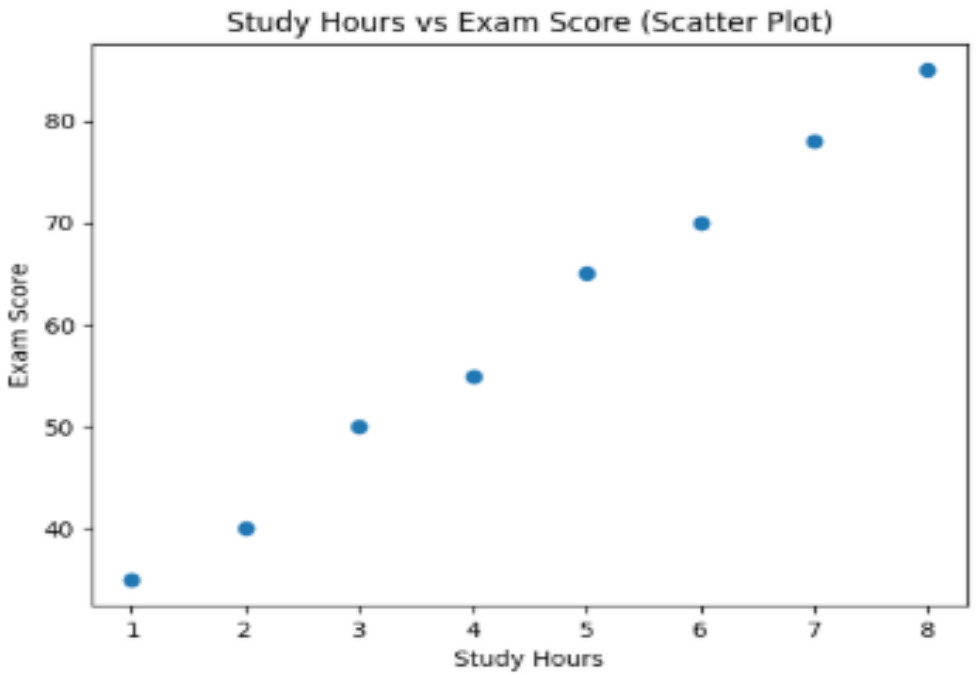
```
Matrix:
 [[10 20 30]
 [40 50 60]
 [70 80 90]]

Element at [1,2]: 60

First Row: [10 20 30]
Second Column: [20 50 80]

Top-left 2x2 Sub-matrix:
 [[10 20]
 [40 50]]
```

MATPLOTLIB(PLOTTING LIBRARY)

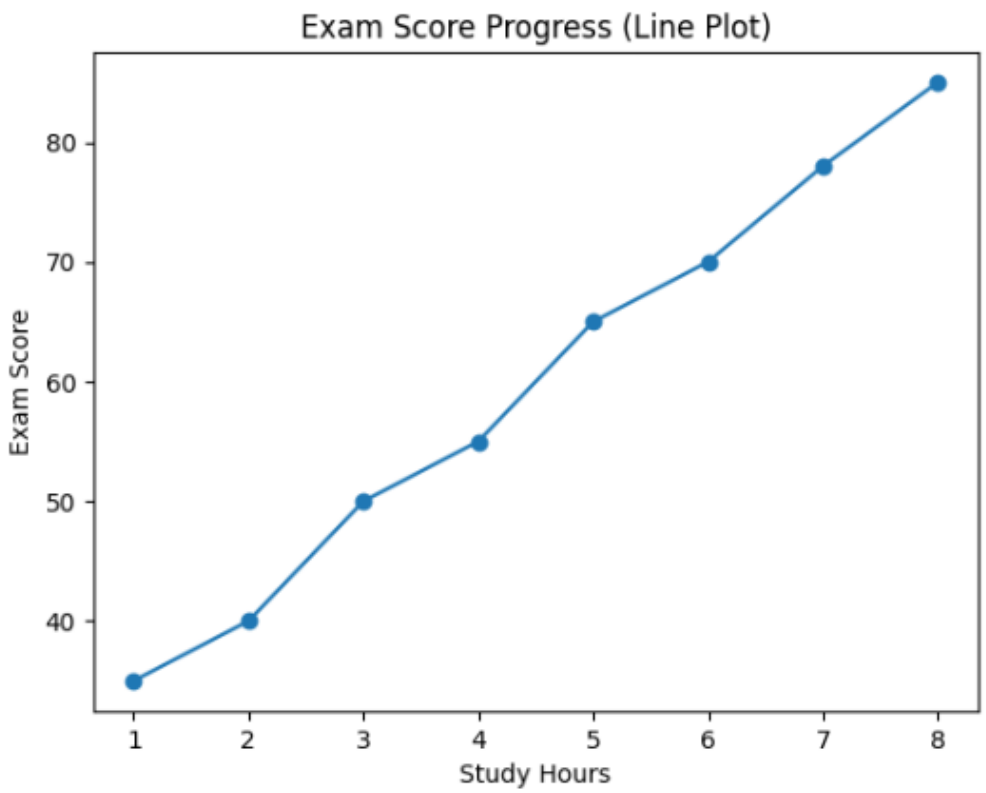
```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
    'ExamScore': [35, 40, 50, 55, 65, 70, 78, 85]
    }

df = pd.DataFrame(data)
print(df)
#T.SIDDARTHA
#REG.NO:11249A373
```

```
   StudyHours  ExamScore
0           1         35
1           2         40
2           3         50
3           4         55
4           5         65
5           6         70
6           7         78
7           8         85
```

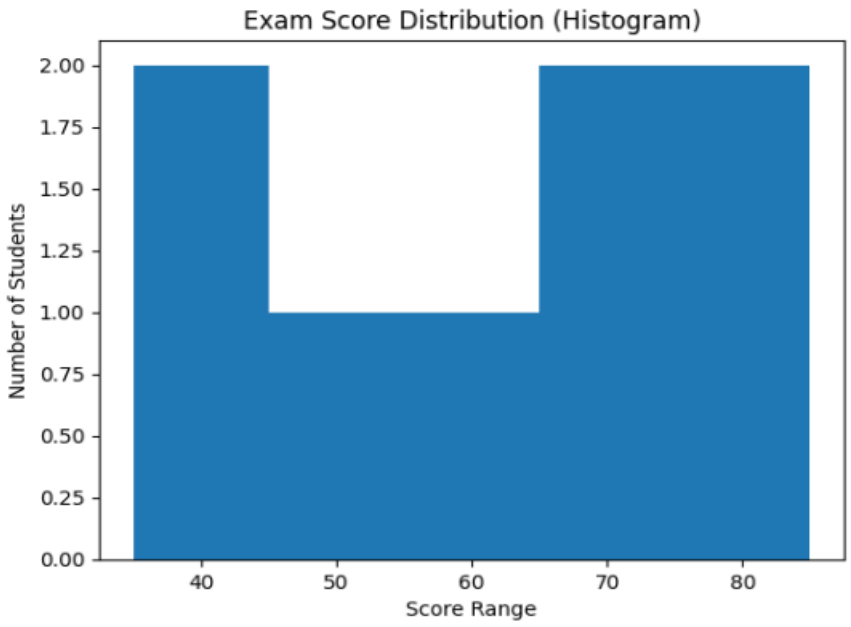
```python
# 1 scatter Plot
plt.scatter(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Scatter Plot)")
plt.show()
#T.SIDDARTHA
#REG.NO:11249A373
```



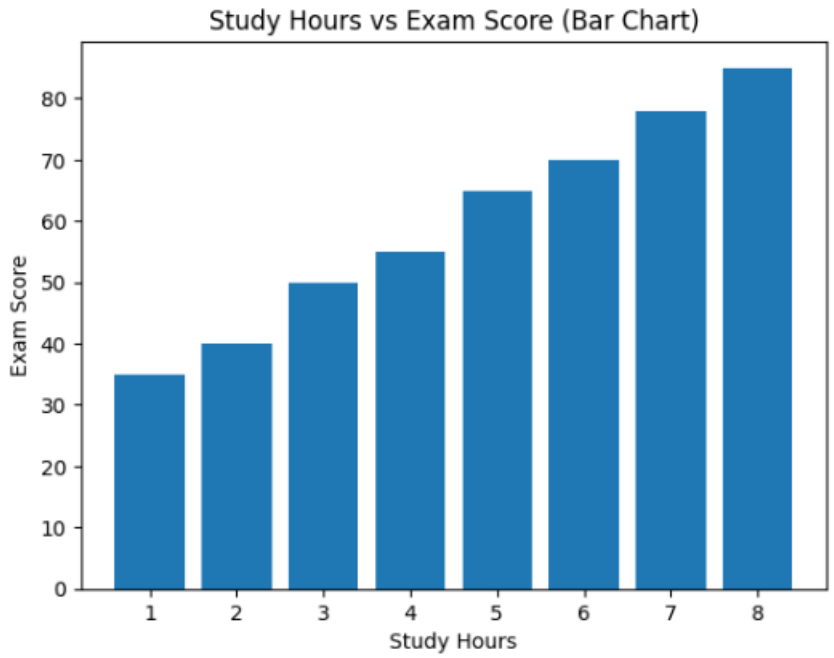
```python
# 2 Line Plot
plt.plot(df['StudyHours'], df['ExamScore'], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress (Line Plot)")
plt.show()
#T.SIDDARTHA
#REG.NO:11249A373
```



```
# 3 Histogram
plt.hist(df['ExamScore'], bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution (Histogram)")
plt.show()
#T.SIDDARTHA
#REG.NO:11249A373
```



```python
# 4 Bar Chart
plt.bar(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Bar Chart)")
plt.show()
```



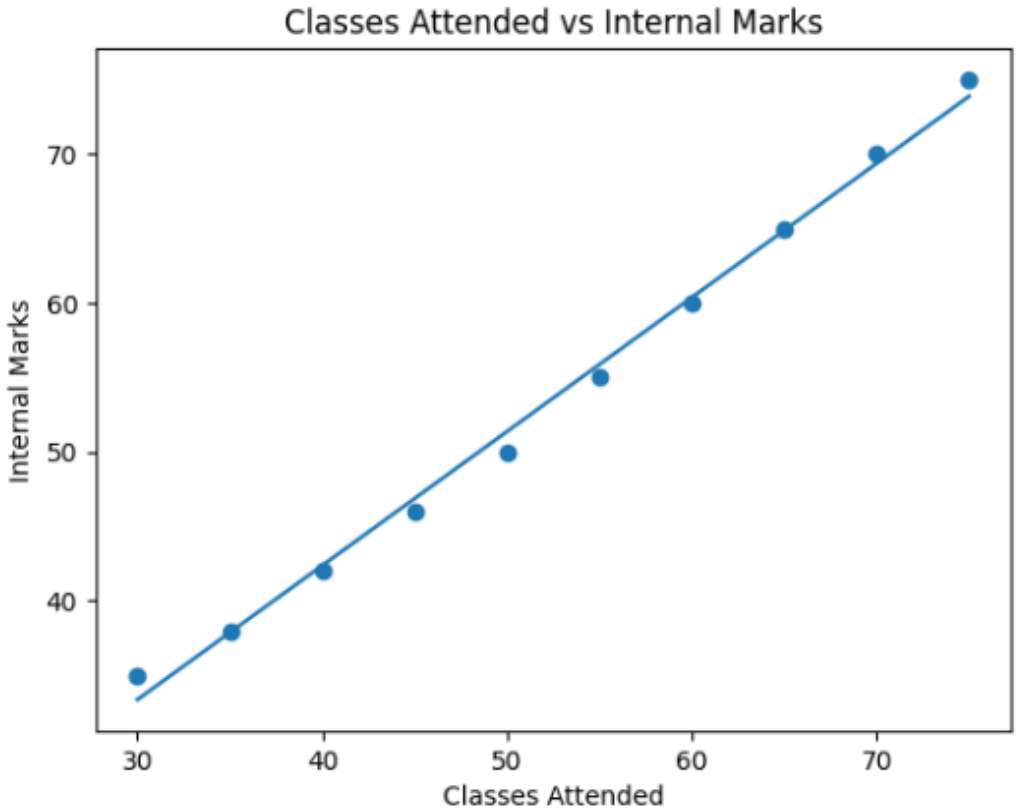
SCIKIT-LEARN (MACHINE LEARNING)

```python
import pandas as pd
df = pd.read_csv('/content/sample_data/california_housing_test.csv')
print(df.head())
print(df.info())
print(df.describe())
#T.SIDDARTHA    #REG.NO:11249A373
```

```
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.05     37.37                27.0       3885.0           661.0
1    -118.30     34.26                43.0       1510.0           310.0
2    -117.81     33.78                27.0       3589.0           507.0
3    -118.36     33.82                28.0         67.0            15.0
4    -119.67     36.33                19.0       1241.0           244.0

   population  households  median_income  median_house_value
0      1537.0       606.0         6.6085            344700.0
1       809.0       277.0         3.5990            176500.0
2      1484.0       495.0         5.7934            270500.0
3        49.0        11.0         6.1359            330000.0
4       850.0       237.0         2.9375             81700.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           3000 non-null   float64
 1   latitude            3000 non-null   float64
 2   housing_median_age  3000 non-null   float64
 3   total_rooms         3000 non-null   float64
 4   total_bedrooms      3000 non-null   float64
 5   population          3000 non-null   float64
 6   households          3000 non-null   float64
 7   median_income       3000 non-null   float64
 8   median_house_value  3000 non-null   float64
dtypes: float64(9)
memory usage: 211.1 KB
None
          longitude    latitude  housing_median_age   total_rooms  \
count  3000.000000  3000.00000         3000.000000   3000.000000
mean   -119.589200    35.63539           28.845333   2599.578667
std       1.994936     2.12967           12.555396   2155.593332
min    -124.180000    32.56000            1.000000      6.000000
25%    -121.810000    33.93000           18.000000   1401.000000
50%    -118.485000    34.27000           29.000000   2106.000000
75%    -118.020000    37.69000           37.000000   3129.000000
max    -114.490000    41.92000           52.000000  30450.000000


       total_bedrooms    population  households  median_income  \
count     3000.000000   3000.000000  3000.00000    3000.000000
mean       529.950667   1402.798667   489.91200       3.807272
std        415.654368   1030.543012   365.42271       1.854512
min          2.000000      5.000000     2.00000       0.499900
25%        291.000000    780.000000   273.00000       2.544000
50%        437.000000   1155.000000   409.50000       3.487150
75%        636.000000   1742.750000   597.25000       4.656475
max       5419.000000  11935.000000  4930.00000      15.000100

       median_house_value
count          3000.00000
mean         205846.27500
std          113119.68747
min           22500.00000
25%          121200.00000
50%          177650.00000
75%          263975.00000
max          500001.00000
```

```python
# STEP 1: IMPORT LIBRARIES
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# STEP 2: CREATE DATASET
data = {
    'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
# STEP 3: FEATURES & TARGET
X = df[['Classes_Attended']]
y = df['Internal_Marks']
# STEP 4: TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7
)
# STEP 5: TRAIN MODEL
model = LinearRegression()
model.fit(X_train, y_train)
# STEP 6: PREDICTION
predictions = model.predict(X_test)
# STEP 7: EVALUATION
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)
# STEP 8: VISUALIZATION
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
#T.SIDDARTHA
#REG.NO:11249A373
```

```
  Student  Classes_Attended  Internal_Marks
0      S1                30              35
1      S2                35              38
2      S3                40              42
3      S4                45              46
4      S5                50              50
5      S6                55              55
6      S7                60              60
7      S8                65              65
8      S9                70              70
9     S10                75              75
MSE: 0.578125
Marks per Class: 0.9
Base Marks: 6.375
```



RESULT: Thus, the python programs using libraries numpy,pandas,matplotlib,scikit has successfully excuted.

EXPERIMENT.NO:02
PERFORM DATA EXPLORATION AND PREPROCESSING IN PYTHON

AIM : To Perform Data Exploration and Preprocessing in python .

PROGRAM :

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

data={
    'Country':['India','USA','India','USA','UK','India'],
    'Age':[22,25,np.nan,30,28,35],
    'Salary':[40000,60000,50000,np.nan,72000,58000],
    'Purchased':['No','Yes','Yes','No','Yes','Yes']
}
df=pd.DataFrame(data)
print("---ORIGINAL RAW DATA---")
print(df)
print("\n")
print("---MISSING VALUES COUNT---")
print(df.isnull().sum())
print("\n")
df['Age']=df['Age'].fillna(df['Age'].mean())
df['Salary']=df['Salary'].fillna(df['Salary'].mean())
print("---DATA AFTER CLEANING---")
print(df)
print("\n")
df_encoded=pd.get_dummies(df,columns=['Country'])
df_encoded['Purchased']=df_encoded['Purchased'].map({'Yes':1,'No':0})
print("---DATA AFTER ENCODING(All Numbers Now)----")
print(df_encoded)
print("\n")
X=df_encoded.drop('Purchased',axis=1)
y=df_encoded['Purchased']
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
print("---FINAL PROCESSED DATA(ready for AI)---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())

#T.SIDDARTHA
#REG.NO:11249A373
```

```
---ORIGINAL RAW DATA---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   India   NaN  50000.0       Yes
3     USA  30.0      NaN        No
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0       Yes


---MISSING VALUES COUNT---
Country      0
Age          1
Salary       1
Purchased    0
dtype: int64


---DATA AFTER CLEANING---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   India  28.0  50000.0       Yes
3     USA  30.0  56000.0        No
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0       Yes


---DATA AFTER ENCODING(All Numbers Now)----
    Age   Salary  Purchased  Country_India  Country_UK  Country_USA
0  22.0  40000.0          0           True       False        False
1  25.0  60000.0          1          False       False         True
2  28.0  50000.0          1           True       False        False
3  30.0  56000.0          0          False       False         True
4  28.0  72000.0          1          False        True        False
5  35.0  58000.0          1           True       False        False


---FINAL PROCESSED DATA(ready for AI)---
        Age    Salary  Country_India  Country_UK  Country_USA
0 -1.484615 -1.644453            1.0   -0.447214    -0.707107
1 -0.742307  0.411113           -1.0   -0.447214     1.414214
2  0.000000 -0.616670            1.0   -0.447214    -0.707107
3  0.494872  0.000000           -1.0   -0.447214     1.414214
4  0.000000  1.644453           -1.0    2.236068    -0.707107
```

RESULT: Thus, the python program to perform data exploration & preprocessing has successfully excuted.

EXPERIMENT.NO:03

IMPLEMENT REGULARIZED LINEAR REGRESSION

AIM :To implement regularized linear regression in python.

```
model_l1 = LogisticRegression(penalty='l1', solver='liblinear', C=1.0)
model_l1.fit(X_train, y_train)

y_pred_l1 = model_l1.predict(X_test)



print("\n--- L2 REGULARIZATION ---")
print("Accuracy:", accuracy_score(y_test, y_pred_l2))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_l2))
print("Classification Report:\n", classification_report(y_test, y_pred_l2))

print("\n--- L1 REGULARIZATION ---")
print("Accuracy:", accuracy_score(y_test, y_pred_l1))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_l1))
print("Classification Report:\n", classification_report(y_test, y_pred_l1))



print("\nL2 Coefficients:\n", model_l2.coef_)
print("\nL1 Coefficients:\n", model_l1.coef_)



plt.figure()
plt.bar(range(len(model_l2.coef_[0])), model_l2.coef_[0])
plt.title("L2 Coefficients")
plt.xlabel("Features")
plt.ylabel("Weight")
plt.show()

plt.figure()
plt.bar(range(len(model_l1.coef_[0])), model_l1.coef_[0])
plt.title("L1 Coefficients")
plt.xlabel("Features")
plt.ylabel("Weight")
plt.show()


print("\n=== END OF EXPERIMENT ===")
```

RESULT :Thus,the python program to regularized linear regression has successfully excuted .

EXPERIMENT.NO:04

IMPLEMENT NAVIE BAYES CLASSIFIER

AIM: To implement navie bayes classifier.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder          //T BRAHMAJI
from sklearn.metrics import accuracy_score              //11249A369


data = {
    'Feature1': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Feature2': ['A', 'B', 'A', 'C', 'B', 'A', 'C', 'B', 'A', 'C'],
    'Target': [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)

print("Dataset:\n", df.head())



le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])



X = df.iloc[:, :-1]
y = df.iloc[:, -1]



X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

:

RESULT:Thus,the python program to implement navie bayes.

EXPERIMENT.NO:05

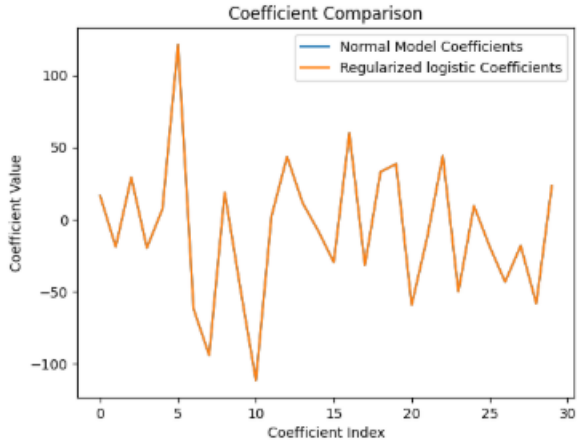
IMPLEMENT REGULARIZED LOGISTIC REGRESSION

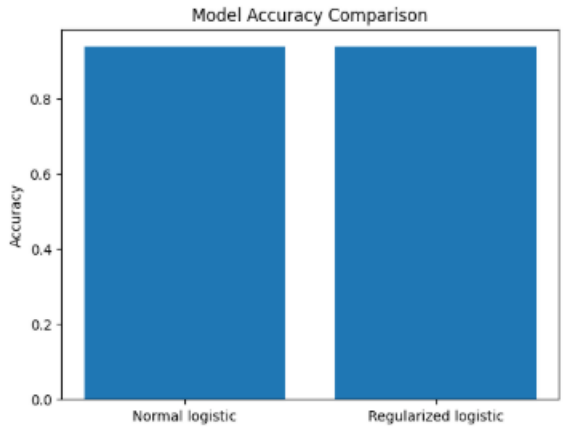
AIM:To perform Regularized Logistic Regression in python .

PROGRAM:

```python
import numpy as np
import pandas as pd                        //T BRAHMAJI
import matplotlib.pyplot as plt             //11249A369

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


print("=== REGULARIZED LOGISTIC REGRESSION ===\n")


df = pd.read_csv("data.csv")

print("First 5 rows:\n", df.head())
print("\nInfo:")
print(df.info())
print("\nMissing values:\n", df.isnull().sum())


for col in df.select_dtypes(include=np.number).columns:
    df[col].fillna(df[col].mean(), inplace=True)

for col in df.select_dtypes(include='object').columns:
    df[col].fillna(df[col].mode()[0], inplace=True)



le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])


X = df.iloc[:, :-1]
y = df.iloc[:, -1]

print("\nFeatures:\n", X.head())
print("\nTarget:\n", y.head())



X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)



scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)



model_l2 = LogisticRegression(penalty='l2', C=1.0, max_iter=200)
model_l2.fit(X_train, y_train)

y_pred_l2 = model_l2.predict(X_test)
```

```
Normal Model Accuracy: 0.9385964912280702
Regularized Model Accuracy: 0.9385964912280702
```





RESULT : Thus, the python program to perform Regularized Logistic Regression has successfully excuted.

EXPERIMENT.NO:06

BUILD MODELS USING DIFFERENT ENSEMBLING TECHNIQUES

AIM: To build models using different ensembling techniques in python .

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split        //T BRAHMAJI
from sklearn.metrics import accuracy_score                  //11249A369


X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([0, 0, 0, 1, 1, 1])


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


model = LogisticRegression(penalty='l2', C=1.0)

model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print("Accuracy:", accuracy_score(y_test, y_pred))


print("Prediction for 7:", model.predict([[7]]))
```

```
Accuracy: 1.0
Prediction for 7: [1]
```

RESULT: Thus, the python program to build models using different ensembling techniques has successfully executed.

EXPERIMENT.NO:07

BUILD MODELS USING DECISION TREES

AIM:To build models using decision trees in python.

PROGRAM:

```
import numpy as np
import pandas as pd                    //T BRAHMAJI
import matplotlib.pyplot as plt         //11249A369

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


print("=== REGULARIZED LOGISTIC REGRESSION ===\n")


df = pd.read_csv("data.csv")

print("First 5 rows:\n", df.head())
print("\nInfo:")
print(df.info())
print("\nMissing values:\n", df.isnull().sum())


for col in df.select_dtypes(include=np.number).columns:
    df[col].fillna(df[col].mean(), inplace=True)

for col in df.select_dtypes(include='object').columns:
    df[col].fillna(df[col].mode()[0], inplace=True)



le = LabelEncoder()
```

RESULT: Thus, the python program to build models using decision trees has successfully executed.

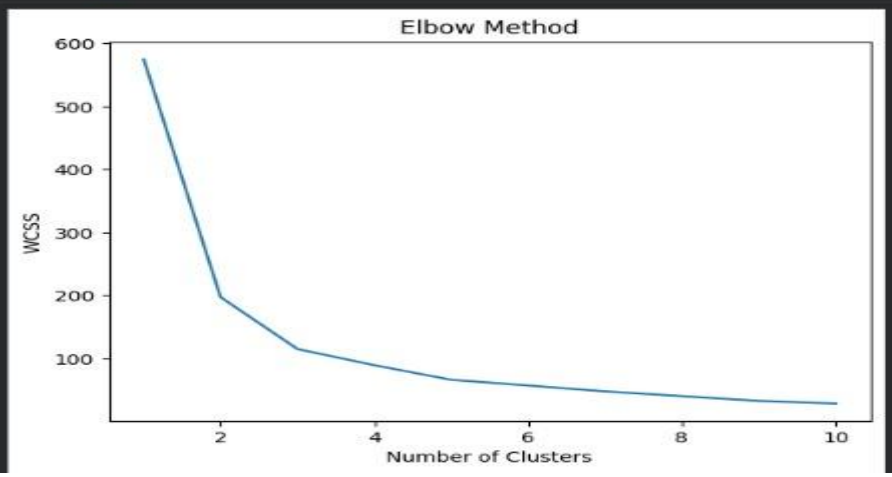
EXPERIMENT.NO:08

SUPPORT VECTOR MACHINE (SVM) WITH DIFFERENT KERNELS

AIM: To perform support Vector Machine (SVM) with different kernels in python.

PROGRAM:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler          //T BRAHMAJI
from sklearn.decomposition import PCA                     //11249A369
from sklearn.cluster import KMeans
data = load_iris()
X = data.data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)


plt.figure()
plt.plot(range(1, 11), wcss)
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()


kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X_pca)


plt.figure()
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```



RESULT: Thus, the python program to support Vector Machine (SVM) with different kernels has successfully executed.

EXPERIMENT.NO:09

IMPLEMENT K-NEAREST NEIGHBOURSCLASSIFICATION

AIM: To perform K- Nearest Neigbours in python.

PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score        //T BRAHMAJI

data = load_iris()                                //11249A369
X = data.data
y = data.target


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)


y_pred = knn.predict(X_test)


print("Accuracy:", accuracy_score(y_test, y_pred))


print("Prediction for [5.1, 3.5, 1.4, 0.2]:",
      knn.predict([[5.1, 3.5, 1.4, 0.2]]))
```

```
Accuracy: 1.0
Prediction for [5.1, 3.5, 1.4, 0.2]: [0]
```

Columns in the dataset: ['id', 'battery_power', 'blue', 'clock_speed', 'dual_sim', 'fc', 'four_g', 'int_memory', 'm_dep', 'mobile_wt', 'n_cores', 'pc', 'px_height', 'px_width', 'ram', 'sc_h', 'sc_w', 'talk_time', 'three_g', 'touch_screen', 'wifi']

Accuracy: 0.465
Confusion Matrix:
[[39 56]
[51 54]]

Classification Report:

	precision	recall	f1-score	support
0	0.43	0.41	0.42	95
1	0.49	0.51	0.50	105
accuracy			0.47	200
macro avg	0.46	0.46	0.46	200
weighted avg	0.46	0.47	0.46	200

RESULT: Thus, the python program to K- Nearest Neigbours has successfully executed.

EXPERIMENT.NO:10

K-MEANS CLUSTERING WITH PCA AND ELBOW METHOD

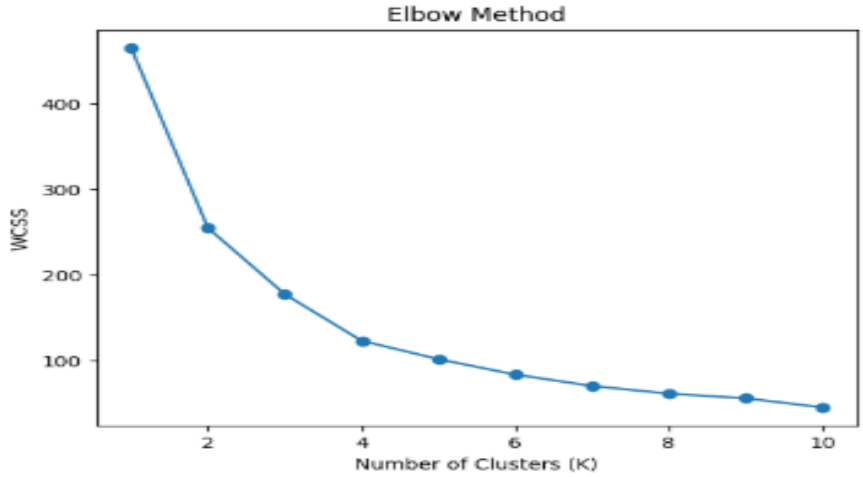
AIM: To perform K-Means Clustering with PCA &Elbow method in python.

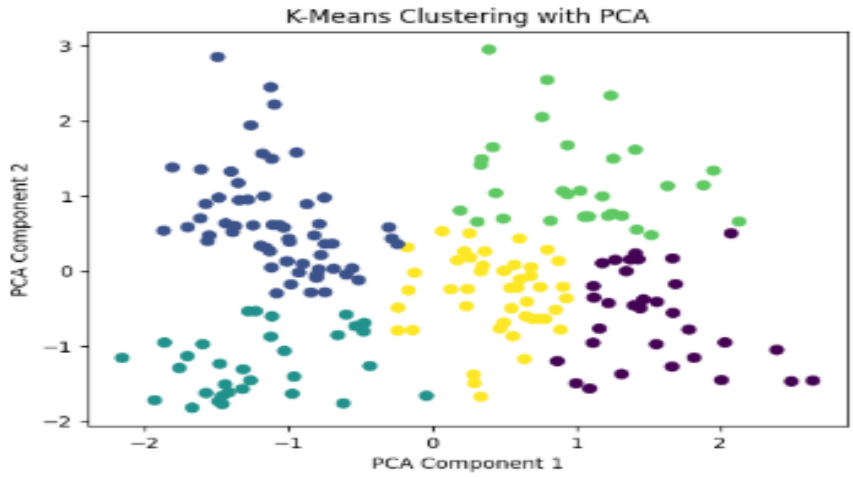
PROGRAM:

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split    //T BRAHMAJI
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score              //11249A369
data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

linear_model = SVC(kernel='linear')
linear_model.fit(X_train, y_train)
pred_linear = linear_model.predict(X_test)

poly_model = SVC(kernel='poly', degree=3)
poly_model.fit(X_train, y_train)
pred_poly = poly_model.predict(X_test)

rbf_model = SVC(kernel='rbf')
rbf_model.fit(X_train, y_train)
pred_rbf = rbf_model.predict(X_test)

print("Linear Kernel Accuracy:", accuracy_score(y_test, pred_linear))
print("Polynomial Kernel Accuracy:", accuracy_score(y_test, pred_poly))
print("RBF Kernel Accuracy:", accuracy_score(y_test, pred_rbf))
```

```
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 1.0
```

```
Explained Variance Ratio: [0.44266167 0.33308378]
```





RESULT: Thus, the python program to K-Means Clustering with PCA &Elbow method has successfully executed.